

PostgreSQL vs MongoDB

Comparison between Relational DBMS and NoSQL Tool

Authors: Sara Santilli 2203141 - Martina Stivala 2192706

Course: Data Management 2024/2025

Sapienza University of Rome

Project Objectives



Core Goal

Evaluate performance and usability differences between a normalized relational DBMS (PostgreSQL) and a denormalized NoSQL Document Database (MongoDB).



Evaluation Criteria

- Query complexity (SQL vs MQL)
- Execution performance (Speed)
- Data consistency & Integrity
- Schema flexibility & Evolution

Dataset: Kaggle – IMDb & TMDb Movie Metadata

Over 1,000,000 records of movies and TV shows including metadata: title, cast, genres, ratings, revenue, and spoken languages.

1M+

Movie Records

[Dataset link](#)

Data cleaning

The clean_movie_dataset.py script handles noise and inconsistencies before DB population:

- Removal of irrelevant columns
- Removal of duplicate rows (by movie ID)
- Removal of rows with missing title, year, rating, genres
- Conversion of year / revenue / budget to int
- Filtered films before 1888 for validation
- Revenue and budget values below 1000 were considered invalid

Data Cleaning script

```
newdf = newdf.drop_duplicates(subset=['id'], keep='first')
newdf = newdf.dropna(subset=["title", "release_year",
                             "vote_average", "genres_list"])
newdf.loc[newdf['revenue'] < 1000, 'revenue'] = np.nan
newdf.loc[newdf['budget'] < 1000, 'budget'] = np.nan
```

PostgreSQL Relational Design

A strictly **Normalized** approach to ensure data integrity and reduce redundancy.

- **Main tables** = movies, genres, actors, writers, keywords, languages
- **Relationship Tables** = movie_genres, movie_actors, movie_writers, movie_keywords

12

Total Tables

```
CREATE TABLE genres (  
    genre_id SERIAL PRIMARY KEY,  
    name TEXT UNIQUE  
);
```

```
CREATE TABLE movie_genres (  
    movie_id INT REFERENCES movies(id),  
    genre_id INT REFERENCES genres(genre_id),  
    PRIMARY KEY(movie_id, genre_id)  
);
```

```
CREATE TABLE movies (  
    id INT PRIMARY KEY,  
    title TEXT NOT NULL,  
    vote_average NUMERIC,  
    vote_count INT,  
    release_date DATE,  
    release_year INT,  
    revenue NUMERIC,  
    budget NUMERIC,  
    runtime INT,  
    popularity NUMERIC,  
    overview TEXT,  
    imdb_id TEXT,  
    language_id INT REFERENCES languages(language_id)  
);
```

MongoDB Document Model

Document Structure Example

```
{  
  "_id": 27205,  
  "title": "Inception",  
  "genres": ["Action", "Sci-Fi", "Adventure"],  
  "stars": ["Leonardo DiCaprio", "Joseph Gordon-Levitt"],  
  "imdb_rating": 8.8,  
  "cast_list_clean": [...],  
  "keywords_list": ["dream", "subconscious"]  
}
```

A **Denormalized** approach prioritizing read speed and development flexibility.

- **Document-oriented structure:** Each movie is stored as a single BSON document
- **Embedded Arrays:** Genres, Cast, and Keywords stored inside the document.
- **Indexing:** Created indices on title, genres, and ratings for optimized filtering.
- Flexible schema
- No JOINS required

Query 1: Single Movie Retrieval

Goal: Retrieve full details of movie Inception (ID: 27205)

PostgreSQL (SQL)

Requires multiple JOINS and Aggregates data

```
SELECT
  m.title,
  l.iso_code AS language,
  r.imdb_rating,
  c.director,
  STRING_AGG(DISTINCT g.name, ', ') AS genres,
  STRING_AGG(DISTINCT a.name, ', ') AS cast_list,
  STRING_AGG(DISTINCT w.name, ', ') AS writers,
  STRING_AGG(DISTINCT k.name, ', ') AS keywords
FROM movies m
LEFT JOIN languages l ON m.language_id = l.language_id
LEFT JOIN movie_ratings r ON m.id = r.movie_id
LEFT JOIN movie_crew c ON m.id = c.movie_id
LEFT JOIN movie_genres mg ON m.id = mg.movie_id
LEFT JOIN genres g ON mg.genre_id = g.genre_id
LEFT JOIN movie_actors ma ON m.id = ma.movie_id
LEFT JOIN actors a ON ma.actor_id = a.actor_id
LEFT JOIN movie_writers mw ON m.id = mw.movie_id
LEFT JOIN writers w ON mw.writer_id = w.writer_id
LEFT JOIN movie_keywords mk ON m.id = mk.movie_id
LEFT JOIN keywords k ON mk.keyword_id = k.keyword_id
GROUP BY m.id, m.title, l.iso_code, r.imdb_rating, c.director;
```

MongoDB (MQL)

use a single find() operation

```
db.movies.find(
  { "_id": 27205 },
  {
    "title": 1,
    "release_year": 1,
    "imdb_rating": 1,
    "director": 1,
    "genres": 1,
    "cast_list_clean": 1,
    "keywords_list": 1
  }
)
```

Query 2: Filtering by Genre, Keyword and Rating

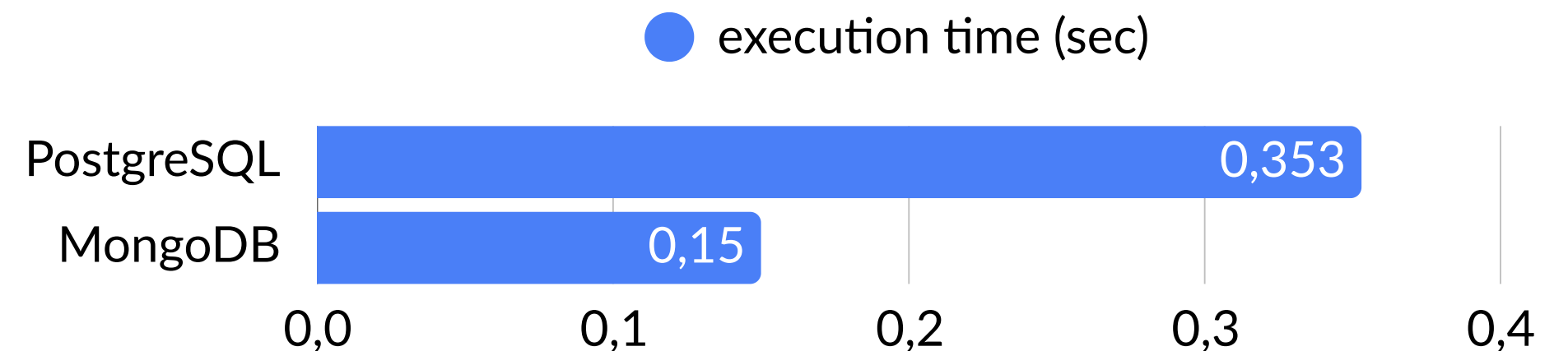
Goal: Find Sci-Fi or Action movies with keyword "space" and Rating > 8

PostgreSQL

```
SELECT m.title, m.popularity, r.imdb_rating
FROM movies m
JOIN movie_ratings r ON m.id = r.movie_id
WHERE r.imdb_rating > 8.0
  AND m.id IN (
    SELECT mg.movie_id FROM movie_genres mg
    JOIN genres g ON mg.genre_id = g.genre_id
    WHERE g.name IN ('Sci-Fi', 'Action')
  )
  AND m.id IN (
    SELECT mk.movie_id FROM movie_keywords mk
    JOIN keywords k ON mk.keyword_id = k.keyword_id
    WHERE k.name = 'space'
  )
ORDER BY m.popularity DESC;
```

MongoDB

```
db.movies.find({
  "imdb_rating": { $gt: 8.0 },
  "genres": { $in: ["Sci-Fi", "Action"] },
  "keywords_list": "space"
})
.sort({ "popularity": -1 })
```



Query 3: Actor Collaborations

Goal: Find actors who worked together most frequently

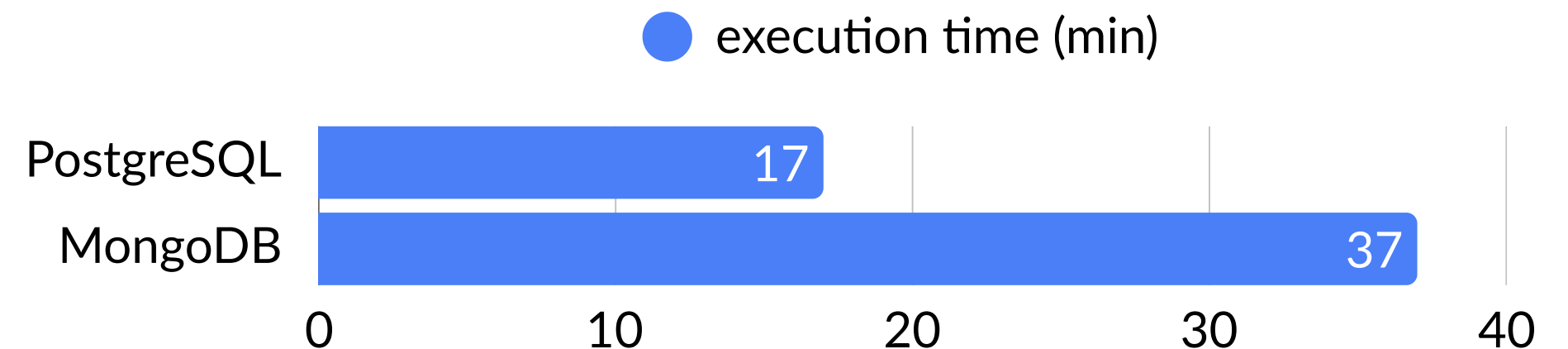
```
SELECT
  a1.name AS actor_1,
  a2.name AS actor_2,
  COUNT(*) AS movies_together
FROM movie_actors ma1
JOIN movie_actors ma2
  ON ma1.movie_id = ma2.movie_id
  AND ma1.actor_id < ma2.actor_id
JOIN actors a1 ON ma1.actor_id = a1.actor_id
JOIN actors a2 ON ma2.actor_id = a2.actor_id
GROUP BY a1.name, a2.name
HAVING COUNT(*) > 1
ORDER BY movies_together DESC
```

PostgreSQL:

- Self JOIN on movie_actors
- GROUP BY aggregation

MongoDB:

- Aggregation pipeline
- Uses array processing



Query 4: Full-Text Search Efficiency

Goal: Search for movies containing “space” and “war”

PostgreSQL

```
CREATE INDEX idx_movies_overview_fts
ON movies
USING gin(to_tsvector('english', overview));
```

```
SELECT
  m.title,
  m.release_year,
  ts_rank(to_tsvector('english', m.overview), query) AS score
FROM movies m,
  to_tsquery('english', 'space & war') query
WHERE to_tsvector('english', m.overview) @@ query
ORDER BY score DESC
LIMIT 10;
```

MongoDB

```
db.movies.createIndex({ "overview": "text" });
```

```
cursor = collection.find(
    {"$text": {"$search": '\"space\" \"war\"'}},
    {
        "title": 1,
        "release_year": 1,
        "score": {"$meta": "textScore"},
        "_id": 0
    }
).sort([("score", {"$meta": "textScore"})]).limit(10)

results = list(cursor)
```

Flexibility Comparison: schema evolution

Goal: Add a new field “youtube_trailer”

PostgreSQL

```
ALTER TABLE movies  
ADD COLUMN youtube_trailer TEXT;
```

- schema modification required
- possible table locking
- NULL values added to all rows

MongoDB

```
result = collection.update_one(  
    { "_id": 27205 },  
    {  
        "$set": {  
            "youtube_trailer":  
                "https://www.youtube.com/watch?v=YoHD9XEInc0"  
        }  
    }  
)
```

- dynamic fields
- immediate update



MongoDB more flexible

Data Consistency

Actor Name Update: Brad Pitt → William Brad Pitt

PostgreSQL

```
UPDATE actors
SET name = `William Brad Pitt`
WHERE name = `Brad Pitt`;
```

- single row update
- normalization guarantees consistency
- no duplicated data

MongoDB

```
result = collection.update_many(
    {"cast_list_clean": "Brad Pitt"},
    {
        "$set": {
            "cast_list_clean.$[elemento]":
                "William Brad Pitt"
        }
    },
    array_filters=[{"elemento": "Brad Pitt"}]
)
```

- multiple document updates
- high I/O cost
- denormalized redundancy



PostgreSQL stronger consistency mode

Final conclusions and Insight

PostgreSQL is better for:

- Complex analytical queries (aggregations)
- Analytical workloads
- Data consistency
- Structured modeling

MongoDB is better for:

- flexible schema
- fast retrieval
- document-based applications
- read-heavy workloads

"No universally superior DBMS exists. The choice depends on data structure, query type, performance needs, scalability requirements."

Thank you for your attention!

Sara Santilli (2203141) & Martina Stivala (2192706)



GitHub Repo